Project Aurora-927 — Security & Deployment Assessment

Technical Engineering Report

DRAFT - For engineering review; site verification required

Generated 05 November 2026 (UTC)

Contents

1. Executive Summary 3

2. Scope, Assumptions, and Assessment Method 3

3. System Overview and Component Responsibilities 4

4. Authentication Flow and Keycloak Integration 5

5. JWT Issuance, Validation, and Token Handling 6

6. API Access Controls and Authorization Model 6

7. Database Security and PostgreSQL Protection 7

8. Deployment Architecture and Trust Boundaries 8

9. Risk Assessment Table: Eight Hypothetical Security Risks and Proposed Mitigations 10

10. Deployment Security Checklist 12

11. Final Recommendations and Review Priorities 13

12. Document Conventions, Evidence Limitations, and Page Numbering 14

1. Executive Summary

This report presents a security and deployment assessment for Project Aurora-927, a fictional application composed of a React frontend, a Django backend, Keycloak for identity and access management, and PostgreSQL for persistent data storage. The assessment focuses on the security relationships among these components, including authentication flows, JSON Web Token (JWT) issuance and validation, API authorization, database protection, deployment boundaries, and operational controls. It is intended to support technical review and deployment planning rather than to certify a production implementation.

The proposed architecture can provide a sound foundation when responsibilities are clearly separated. Keycloak should act as the authoritative identity provider and token issuer; the Django application should validate tokens and enforce application-specific authorization; React should handle user-facing authentication interactions without being treated as a trusted security boundary; and PostgreSQL should remain accessible only through controlled backend connections. Secure deployment depends on consistent configuration across these layers, including protected transport, validated token claims, narrowly scoped permissions, secure secret handling, and restricted network exposure.

No implementation evidence, test output, configuration export, source code, or infrastructure records were provided for this assessment. Accordingly, this report does not assert that any vulnerability has been confirmed, and it does not present fabricated measurements or test results. Security concerns identified in the broader assessment should be interpreted as hypothetical risks or review priorities until verified against the actual application and deployment environment. Particular attention should be given to issuer and audience validation for JWTs, token lifetime and storage decisions, role and scope enforcement at API endpoints, protection of administrative interfaces, and database credentials and connectivity.

The principal security objective is to establish an explicit chain of trust from the user's authentication event through API request authorization and database access. Each service should accept only the inputs and privileges necessary for its role. Authentication should be centralized in Keycloak, while authorization decisions should be enforced server-side by Django and should not rely solely on client-side controls or user-supplied claims. PostgreSQL should not be directly exposed to browsers or public networks, and deployment components should be isolated according to their trust requirements.

Recommended review priorities are to document the intended authentication and authorization model, verify Keycloak client and realm configuration, confirm Django's JWT validation behavior, inventory API permissions, and define secure deployment boundaries. Before production release, the team should also complete the deployment checklist, validate secret and certificate management, establish logging and monitoring requirements, and perform independent security testing against the implemented system.

2. Scope, Assumptions, and Assessment Method

This assessment covers the proposed Project Aurora-927 application and its principal security and deployment concerns across the React frontend, Django backend, Keycloak identity provider, PostgreSQL database, and supporting deployment infrastructure. The review considers authentication, token issuance and validation, API authorization, secrets management, network exposure, database protection, logging, and operational controls. It addresses both application design and the interfaces between components, because security failures frequently arise at trust boundaries rather than within a single service.

The assessment is limited to the architecture and behaviors described or implied by the project scope. No source code, configuration files, infrastructure manifests, runtime logs, penetration-test output, dependency inventory, or production telemetry has been provided. Accordingly, this document does not confirm vulnerabilities, compliance status, performance characteristics, or the effectiveness of any control. Items identified as risks are hypothetical conditions that should be verified during implementation review, testing, and deployment validation.

The following assumptions establish the assessment context. Keycloak is the authoritative identity provider, Django exposes protected API endpoints, React operates as the user-facing client, and PostgreSQL is accessible only through the backend rather than directly from browsers or untrusted networks. TLS is expected for external and service-to-service communications where credentials, tokens, or sensitive data may be transmitted. Environment-specific configuration, signing keys, database credentials, and other secrets are assumed to be managed outside application source code. These assumptions require confirmation, particularly for deployment topology, token storage, administrative access, backup handling, and network segmentation.

The method is a structured, design-focused review rather than a penetration test or formal audit. First, the system boundaries and trust relationships are identified. Next, expected authentication and authorization flows are examined, including the creation, transport, validation, expiration, and revocation implications of JWTs. The review then considers component configuration responsibilities, failure modes, privilege separation, data exposure, and operational safeguards. Finally, the findings are expressed as implementation checks, hypothetical risks, and prioritized recommendations suitable for technical review.

Evidence limitations are explicitly retained throughout the report. Statements describing intended controls are architectural expectations, not evidence that those controls are currently implemented. Where a control cannot be verified from the available material, the appropriate action is to validate it through code inspection, configuration review, controlled testing, or deployment evidence. The assessment therefore emphasizes defensible design decisions and verification priorities without assigning unsupported severity ratings or claiming observed test results.

3. System Overview and Component Responsibilities

Project Aurora-927 is assessed as a web application composed of a React front end, a Django-based application and API layer, Keycloak for identity and access management, and PostgreSQL for persistent data storage. The components are logically separated by function and should communicate through explicitly defined interfaces. This separation supports independent security controls, limits the impact of a compromise, and provides clear ownership of authentication, authorization, business processing, and data protection responsibilities.

The React application is the client-facing presentation layer. It renders user interfaces, collects input, initiates authentication, and calls Django API endpoints over HTTPS. The client should treat all data received from the browser as untrusted, including route parameters, form values, and locally stored token material. React may hide user interface elements based on permissions, but such behavior is only a usability feature; it must not be treated as an authorization control. The front end is also responsible for avoiding unsafe rendering patterns, minimizing exposure of credentials or tokens, and handling session expiration without disclosing sensitive information.

Django provides the principal application and API boundary. It should validate request structure, enforce authentication requirements, apply authorization decisions, execute business rules, and return only the data appropriate to the requesting principal. Django is responsible for validating Keycloak-issued tokens according to the configured issuer, signature, audience, expiry, and relevant claims. It should also enforce server-side protections such as input validation, CSRF controls where applicable, rate limiting, secure error handling, and consistent logging. Django must not rely solely on claims supplied by the browser or on access restrictions implemented in React.

Keycloak is the centralized identity provider and token issuer. It manages realms, clients, users, credentials, authentication policies, roles, groups, and token lifetimes. Keycloak should authenticate users and issue tokens for the intended application context, while Django remains responsible for interpreting those tokens and enforcing API permissions. Administrative access to Keycloak is a separate security concern and should be restricted, monitored, and protected with strong administrative authentication. Configuration of redirect URIs, client types, scopes, and signing keys directly affects the trust relationship between the front end, API, and identity provider.

PostgreSQL is the system of record for application data. It should be reachable only by authorized application services through a controlled network path, use dedicated credentials with least privilege, and protect connections and stored data according to the deployment environment’s requirements. Database authorization does not replace application-level authorization: Django must ensure that queries are scoped to the authenticated user, tenant, or business role before data is returned or modified. Backups, migrations, audit records, and operational secrets should be managed as part of the deployment security design rather than treated as incidental infrastructure.

4. Authentication Flow and Keycloak Integration

Project Aurora-927 should use Keycloak as the central identity provider and authorization server, with the React client acting as a public application and Django serving as the protected resource server. The recommended protocol is OpenID Connect (OIDC) over TLS, using the Authorization Code Flow with Proof Key for Code Exchange (PKCE). This design avoids exposing client secrets in the browser and reduces the risk associated with intercepted authorization codes.

A typical authentication sequence begins when an unauthenticated user accesses a protected React route. The client redirects the user to the Keycloak authorization endpoint, specifying the configured realm, client identifier, redirect URI, requested scopes, response type, and PKCE challenge. Keycloak authenticates the user according to its configured policies, which may include passwords, multifactor authentication, federation, or account verification. After successful authentication and consent where applicable, Keycloak returns an authorization code to the registered redirect URI. The React application exchanges this code through the appropriate OIDC mechanism and receives an access token, with an ID token and refresh token issued according to the client and session configuration.

The access token is presented by the client in the HTTP Authorization header using the Bearer scheme when calling Django APIs. Django should not treat successful authentication by the React application as sufficient evidence of authorization. Instead, the API must independently validate the token and apply endpoint-specific permissions. The ID token is intended primarily for client-side identity information and should not be used as the sole credential for API access. Refresh tokens should be handled as sensitive credentials, with storage and renewal behavior selected to limit exposure in browser environments.

Keycloak client configuration is a critical integration boundary. The client should be configured with exact redirect URIs and permitted web origins rather than broad wildcards. Production and non-production environments should use separate realms or clients, credentials, redirect settings, and signing keys where practical. Public browser clients must not contain a client secret. Administrative Keycloak interfaces and service-account clients should be isolated from ordinary user-facing configuration and granted only the privileges required for their functions.

Logout should address both the local application session and the Keycloak session where centralized sign-out is required. The application should clear locally held tokens and cached identity data, while Keycloak session termination should be initiated through the supported OIDC logout mechanism. Token expiration, refresh failure, revoked sessions, and account disablement should result in predictable reauthentication rather than indefinite access.

The available project description does not establish that these settings are implemented or correctly configured. Accordingly, the flow described here is the recommended integration model and assessment baseline, not evidence of a confirmed implementation or vulnerability. Configuration review should verify redirect restrictions, PKCE enforcement, issuer selection, TLS protection, session policies, multifactor requirements, and separation of administrative access.

5. JWT Issuance, Validation, and Token Handling

JWT issuance should be centralized in Keycloak rather than implemented by the React client or Django application. After successful authentication, Keycloak issues an access token containing the configured issuer, subject, audience, expiration, and authorization claims. The client should request only the scopes and roles required for the intended API operations. Access tokens should be short-lived, while refresh tokens should be handled according to the deployment's session and revocation requirements. Token lifetimes, client types, redirect URIs, and allowed flows should be explicitly configured and reviewed for each environment.

The React application should treat access tokens as sensitive credentials. Where practical, tokens should be kept out of browser persistent storage to reduce exposure through client-side compromise or cross-site scripting. A controlled in-memory approach, combined with a secure session design, is preferable to unrestricted use of local storage. If refresh tokens are used in a browser-based flow, the design should use an approved authorization-code flow with PKCE and should consider a backend-for-frontend or secure, HttpOnly cookie pattern. Tokens must not be placed in URLs, exposed in error messages, or transmitted over non-TLS connections.

Django must validate every bearer token at the API boundary before processing protected requests. Validation should include signature verification using Keycloak's published signing keys, issuer verification against the expected realm, audience verification for the API, and checks for token expiration and other applicable time constraints. The API should accept only explicitly approved signing algorithms and should reject unsigned tokens, malformed tokens, tokens signed with an unexpected algorithm, and tokens issued by another realm or environment. Key material should be obtained through a trusted JWKS endpoint and cached with a controlled refresh strategy so that key rotation can occur without requiring application redeployment.

Successful authentication does not by itself authorize an operation. Django should interpret validated claims consistently and enforce endpoint-specific permissions using roles, scopes, or both. Claims such as subject, client identifier, and authenticated-user attributes should be used only for their defined purposes, with server-side checks preventing users from selecting another user's resource through request parameters. The application should not trust client-supplied identity headers or decoded, but unverified, token contents.

Operational handling should minimize token exposure. Authorization headers, request bodies, query strings, and decoded token payloads should be excluded from application and proxy logs. Monitoring may record limited identifiers and validation outcomes, subject to privacy requirements. Revocation expectations should be documented, recognizing that already-issued access tokens generally remain usable until expiration unless additional introspection or deny-list controls are implemented. Any deviations from these practices should be documented as design decisions for Project Aurora-927, not treated as confirmed vulnerabilities.

6. API Access Controls and Authorization Model

API authorization should follow a deny-by-default model in which every protected request is evaluated against the authenticated subject, the requested operation, and the target resource. Successful JWT validation establishes the caller's identity and token validity, but it should not by itself grant access to all API functions. Django views, viewsets, and service-layer methods should apply explicit permission checks before reading or modifying application data. Public endpoints should be limited to functions that genuinely require anonymous access, such as health checks or selected authentication-related operations, and should expose no sensitive implementation or operational details.

Keycloak should provide the central identity and role-management function, while the Django API should enforce the resulting authorization decisions. Realm roles, client roles, or mapped claims may represent broad capabilities such as administrator, operator, analyst, or ordinary user. The API should validate that claims are intended for the API client and should use the minimum set of roles or scopes necessary for each endpoint. Role names and claim mappings should be documented and version-controlled so that a change in Keycloak configuration does not

silently alter application privileges. Administrative roles should be assigned sparingly, with separate roles for read-only oversight and state-changing operations where practical.

Coarse-grained roles are insufficient for resources that belong to individual users, teams, or organizational units. The application should perform object-level authorization after locating the requested resource and before returning or changing it. For example, a user permitted to view their own records should not gain access merely by changing an identifier in the URL. Querysets should be filtered by the caller's permitted ownership or organizational scope, and update and deletion paths should repeat the authorization decision rather than relying solely on controls applied during retrieval. Bulk operations require equivalent checks for every affected object or should be restricted to explicitly privileged roles.

Authorization controls should also address request context and operation sensitivity. State-changing endpoints should use appropriate input validation, transaction boundaries, and protection against replay or unintended duplicate submissions where relevant. If browser-based sessions or cookies are introduced alongside bearer tokens, CSRF protections must be applied to those session-authenticated requests. CORS should allow only approved front-end origins and should not be used as a substitute for authorization.

Service-to-service access should use dedicated Keycloak clients and narrowly scoped credentials rather than shared human accounts. Secrets must be stored outside source code and rotated according to an established procedure. Authorization failures should return consistent responses that do not disclose whether protected resources exist, while security-relevant decisions should be logged with the subject, resource, action, and outcome. Logs must exclude tokens and other sensitive values. Periodic review should verify role assignments, inactive accounts, privileged endpoints, and alignment between documented policy and implemented permission checks.

7. Database Security and PostgreSQL Protection

PostgreSQL should be deployed as a private data service, reachable only by the Django application and approved administrative paths. It should not be exposed directly to the public network or to browser clients. Network controls should restrict inbound connections to the application tier, while security groups, firewall rules, or equivalent controls should deny all other sources by default. Administrative access should use a controlled management path with strong authentication, auditing, and time-limited privileges.

The database should use dedicated roles with narrowly defined responsibilities. The Django runtime account should have only the permissions required to operate the application schema and should not be granted superuser, replication, or cluster-wide administrative privileges. Schema ownership, migration execution, backup operations, and operational administration should be assigned to separate accounts where practical. Credentials should be stored in a managed secrets facility or protected deployment configuration rather than source code, container images, or client-side configuration. Passwords and connection strings should be rotated through a documented process.

Encryption should be applied both in transit and at rest. TLS should protect PostgreSQL connections between Django and the database, with certificate verification enabled and weak protocol or cipher options disabled according to the hosting environment's supported baseline. Database storage, snapshots, and backup media should use platform-supported encryption with controlled key access. Encryption does not replace authorization: application secrets, database roles, and key-management permissions should remain independently restricted.

Django should access PostgreSQL through parameterized queries and the framework's ORM or an equivalently safe data-access layer. Raw SQL should be reviewed for injection risk, especially where identifiers, ordering clauses, or filters are constructed dynamically. Database error details should not be returned to API clients, because messages may disclose schema names, query structure, or connection information. Production logging should capture useful operational events without recording passwords, tokens, session data, or unnecessary

personal information.

PostgreSQL should be maintained at a supported version and receive security updates through a defined patching process. Configuration should be reviewed for authentication methods, listening interfaces, connection limits, logging, and timeout behavior. Audit logging should be proportionate to operational and privacy requirements, with protected log storage and retention rules. Excessive connection creation can exhaust database resources; connection pooling and sensible limits should therefore be configured and monitored.

Backups should be encrypted, access-controlled, and tested through documented restoration exercises. Retention should reflect business and regulatory requirements, while deletion procedures should address obsolete backup copies. Recovery procedures should identify the required database version, secrets, migration state, and network dependencies. These controls are recommended protections for the fictional Aurora-927 design; no implementation evidence has been provided to confirm their current status or to establish a confirmed database vulnerability.

8. Deployment Architecture and Trust Boundaries

Project Aurora-927 should be deployed as a tiered service rather than exposing application components directly to the public internet. A typical production arrangement places a reverse proxy or load balancer at the internet-facing edge, with HTTPS termination, request routing, and basic traffic controls. The React application may be delivered as static content through the edge tier or a dedicated web server. Browser requests to the Django API should use HTTPS and a controlled API hostname, avoiding mixed-content operation and reducing ambiguity about routing and security policy enforcement.

The Django application should operate in a private application network that is reachable from the edge tier but not directly from external clients. Application instances should be treated as stateless where practical, allowing controlled scaling and replacement without retaining sensitive security state locally. Administrative interfaces, debugging endpoints, health checks, and operational consoles should either be disabled in production or restricted to an administrative network with strong authentication and logging. Secrets, including database credentials, signing configuration, and Keycloak client credentials, should be supplied through an approved secrets-management mechanism rather than committed to source code, container images, or frontend assets.

Keycloak represents a distinct identity trust boundary. Django may trust tokens issued by the configured Keycloak realm only after validating the issuer, signature, audience, expiration, and other applicable claims. Network access from Django to Keycloak should be limited to required protocols and endpoints. The browser should not receive confidential client credentials, and the React application should be configured as a public client only where the selected OAuth flow and deployment model require it. Redirect URIs, allowed origins, and logout destinations should be narrowly defined to prevent an attacker from redirecting authentication responses to an unauthorized site.

PostgreSQL should reside in a separate data network with no public ingress. Only the Django application tier, and narrowly defined administrative tooling where necessary, should be permitted to connect to the database port. Database access should use encrypted connections where supported, dedicated least-privilege roles, and credentials that are rotated independently of application deployment. Backups, replicas, and monitoring services constitute additional trust boundaries and should receive only the access required for their functions. Backup storage must be protected against unauthorized reading and deletion because it may contain personal data, credentials, or application secrets.

Container or host boundaries provide useful isolation but should not be treated as complete security barriers. Each service should run with minimized privileges, restricted filesystem access, controlled egress, and a current base image. Logs crossing tiers should be protected from tampering and should exclude access tokens, passwords, and unnecessary personal data. These boundaries and controls are architectural expectations for review; their

implementation remains an operational matter to be verified during deployment assessment.

9. Risk Assessment Table: Eight Hypothetical Security Risks and Proposed Mitigations

The following risks are hypothetical exposures for the described React, Django, Keycloak, and PostgreSQL architecture. They are not confirmed vulnerabilities or findings from executed tests. Risk ratings reflect potential impact and plausible exposure in a typical deployment; they should be validated through configuration review, threat modeling, and appropriate security testing before remediation priorities are finalized.

ID	Hypothetical risk	Potential impact	Proposed mitigation
R1	Incorrect JWT validation, such as accepting an unintended algorithm, issuer, audience, or signing key	Unauthorized API access or privilege escalation	Configure Django to enforce the expected signing algorithm, issuer, audience, expiration, and key identifier. Retrieve trusted keys through a controlled Keycloak JWKS process and test rejection of malformed, expired, and incorrectly issued tokens.
R2	Inadequate authorization checks after authentication	Users may access another user's records or administrative functions	Enforce authorization server-side for every protected endpoint. Apply role- and scope-based permissions, verify object ownership, and deny access by default. Do not rely on React route restrictions as a security control.
R3	Excessive privileges assigned to Keycloak clients, users, or service accounts	Compromise of one identity could provide broader access than required	Define narrowly scoped roles and client permissions, separate interactive and service identities, review group membership, and remove unused roles. Establish an approval and periodic recertification process for privileged access.
R4	Misconfigured CORS, CSRF, or browser security settings	Cross-origin abuse, unauthorized state-changing requests, or data exposure in browsers	Restrict allowed origins to approved application domains, configure credentials deliberately, retain CSRF protection where cookie-based authentication is used, and apply suitable Content Security Policy, framing, and transport-security headers.
R5	Secrets exposed in source code, container layers, logs, or deployment configuration	Database, Keycloak, or signing-key compromise	Store secrets in a managed secret store or protected runtime configuration, exclude them from repositories and images, rotate credentials, limit access, and redact tokens and passwords from logs and error responses.
R6	PostgreSQL exposed beyond the required private trust boundary	Direct database compromise, data theft, or destructive modification	Place PostgreSQL on a private network, restrict inbound access to approved application identities, require encrypted connections where applicable, use least-privileged database roles, and maintain tested backups with restricted administrative access.

ID	Hypothetical risk	Potential impact	Proposed mitigation
R7	Unpatched dependencies or vulnerable container and operating-system packages	Exploitation of known flaws in application or infrastructure components	Pin and inventory dependencies, scan images and packages during the delivery process, monitor advisories, apply security updates promptly, and rebuild images from maintained base versions.
R8	Insufficient monitoring, rate limiting, or recovery controls	Delayed detection of attacks and prolonged service or data impact	Centralize security-relevant logs, alert on authentication anomalies and repeated authorization failures, apply endpoint-appropriate rate limits, retain audit records securely, and exercise incident-response and backup-restoration procedures.

Residual risk should be reassessed after controls are implemented. Particular attention should be given to token validation, object-level authorization, secret handling, and network isolation because failures in these areas can undermine otherwise effective authentication and deployment controls.

10. Deployment Security Checklist

Before production deployment, confirm that all components are configured according to documented security requirements and that responsibility for each control is assigned. The checklist should be completed for the React frontend, Django application, Keycloak identity service, PostgreSQL database, reverse proxy or gateway, container platform, and supporting infrastructure. Items that cannot be verified should remain open rather than being treated as completed by assumption.

Application and configuration

- Deploy only approved application versions and dependencies, with source changes reviewed and known vulnerable packages addressed through the project’s release process.
- Remove development settings, debug features, test accounts, sample data, administrative utilities, and verbose error responses from production.
- Store secrets outside source control and inject them through an approved secret-management mechanism. Use separate credentials and signing keys for development, testing, and production.
- Set secure values for Django configuration, including allowed hosts, trusted origins, secure cookies, HTTPS enforcement, CSRF protection, and controlled administrative access.
- Configure the frontend to use approved service endpoints and ensure that tokens, credentials, and sensitive responses are not written to browser storage or client-side logs without a documented justification.

Identity and API protection

- Verify that Keycloak realms, clients, redirect URIs, allowed origins, password policies, multifactor requirements, session timeouts, and administrative roles match the approved design.
- Ensure that client secrets are used only by confidential clients and that public clients do not receive privileges intended for backend services.
- Confirm that Django validates JWT signature, issuer, audience, expiration, not-before, and required claims using trusted key material and controlled key-rotation procedures.
- Enforce authorization at every protected API endpoint, including object-level checks that prevent users from accessing another user’s records by changing identifiers.
- Apply rate limits and monitoring to authentication, token, password-reset, administrative, and resource-intensive endpoints. Return errors that support troubleshooting without exposing credentials, tokens, stack traces, or internal topology.

Data, network, and infrastructure

- Restrict PostgreSQL network access to approved application hosts or security groups; disable public exposure and use encrypted connections where supported.
- Use least-privilege database roles, separate migration and runtime permissions, strong credentials, and controlled backup access. Verify that backups are encrypted and subject to defined retention and restoration procedures.

- Segment public, application, identity, and data tiers. Permit only required ports and protocols, and restrict management interfaces to approved administrative networks.
- Terminate TLS with current, approved configurations and protect internal traffic where trust boundaries require it. Configure security headers, access logging, and alerting at the gateway and application layers.
- Run services with non-root identities, read-only filesystems where practical, minimal images, restricted container capabilities, and controlled outbound network access.

Finally, conduct a release-readiness review covering dependency status, secret scanning, configuration review, logging, alert routing, rollback procedures, and incident contacts. Record evidence for each completed item, identify exceptions with an owner and due date, and repeat the checklist after material architecture, identity, or infrastructure changes.

11. Final Recommendations and Review Priorities

Project Aurora-927 should proceed through a controlled security review before production deployment. The highest priority is to verify that authentication, token validation, authorization, and deployment configuration operate as a single, consistently enforced security model. Controls should be implemented at the Django API boundary rather than relying on React behavior or client-side route restrictions. The frontend should be treated as untrusted with respect to authorization decisions.

Keycloak integration should be reviewed first. Confirm that the configured issuer, audience, signing algorithms, key-discovery endpoint, token lifetime, clock-skew handling, and required claims are explicitly defined and consistently validated by the API. Key rotation and key-discovery failure behavior should be documented and exercised in a controlled environment. Refresh-token handling, logout expectations, session expiration, and browser storage choices also require review because weaknesses in these areas can extend the impact of credential or token compromise.

Authorization should be assessed separately from authentication. Each sensitive endpoint should have an identified authorization requirement, including the roles, groups, scopes, or resource ownership checks that apply. Server-side checks should prevent horizontal privilege escalation, unauthorized object access, and administrative actions by ordinary users. Where access decisions depend on database records, those checks should be performed transactionally and should fail closed when identity, ownership, or policy information is missing.

Deployment hardening is the next priority. Production secrets, signing keys, database credentials, and administrative credentials should be supplied through an approved secret-management mechanism and excluded from source control, images, logs, and client bundles. Network exposure should be minimized so that only required public services are internet-accessible; Keycloak administration, PostgreSQL, internal services, and management interfaces should remain restricted. TLS termination, secure headers, cookie attributes, CORS, CSRF protections where applicable, rate limiting, and centralized audit logging should be reviewed as deployment controls rather than assumed from framework defaults.

PostgreSQL protection should include least-privilege accounts, restricted network access, encrypted connections where required, controlled migrations, backup protection, and restoration procedures. Application queries should use parameterized mechanisms and enforce tenant or ownership boundaries at the service and data layers when relevant. Logging should support investigation without recording passwords, tokens, or unnecessary personal data.

Review evidence should distinguish confirmed findings from hypothetical risks identified during design analysis. Before release, the team should maintain an evidence register containing configuration reviews, authorization test cases, dependency and image scans, secret checks, backup-restore validation, and operational sign-offs. Any unresolved high-impact issue should have an owner, compensating control, and approved remediation date. After deployment, security review should recur following identity-provider changes, major dependency upgrades,

authorization-model changes, or material infrastructure redesign.

12. Document Conventions, Evidence Limitations, and Page Numbering

This report uses numbered headings aligned with the approved report outline. Technical terms such as React, Django, Keycloak, PostgreSQL, JWT, API, and TLS are used in their conventional engineering sense. Component names describe the fictional Project Aurora-927 architecture and should not be interpreted as evidence of a specific product configuration unless explicitly stated. “Must” identifies a required control or decision, “should” identifies a recommended practice, and “may” identifies an implementation option dependent on operational constraints.

Security observations are categorized according to the strength of the available evidence. Confirmed findings require direct, verifiable evidence such as configuration data, source code, deployment manifests, logs, test output, or documented operational procedures. Hypothetical risks describe credible failure conditions that could arise if a control is absent, incorrectly configured, or not maintained; they are not assertions that the condition currently exists. Recommendations are proposed actions intended to reduce exposure or improve assurance and should be validated against the final implementation.

The assessment is constrained by the information provided for this fictional system. No claim is made that an active penetration test, code review, configuration audit, vulnerability scan, database inspection, or production deployment review was performed. Accordingly, the report does not present fabricated measurements, test results, exploitability conclusions, compliance determinations, or source citations. Statements about authentication, JWT validation, authorization, database protection, and deployment boundaries describe expected security properties and review priorities rather than verified implementation outcomes.

The risk assessment table presents eight hypothetical security risks and corresponding mitigations. Risk descriptions should be treated as scenarios for design review and deployment verification. Their inclusion does not establish that a vulnerability is present. Before release, each scenario should be mapped to an accountable owner, a verification method, and an appropriate acceptance criterion. Evidence gathered during that process should be retained so that reviewers can distinguish implemented controls from planned controls and inherited platform capabilities.

Page numbers should appear in the footer of each page, using a consistent format such as “Page X of Y” where supported by the document-generation tool. Numbering should begin with the first page of the report body unless organizational requirements specify a separate cover-page convention. Page numbers should remain sequential after revisions, table reflow, or conversion to PDF. Reviewers should rely on section headings and table labels in addition to page references, because pagination can change when fonts, margins, accessibility tags, or PDF rendering settings are modified. The final exported PDF should be checked to confirm that page numbers are visible, sequential, and not obscured by content.